CodeMitVishal
@CodeMitVishal

LIVE SESSION · PYTHON FOR BEGINNERS

Python Foundations

From your first variable to classes, files and exceptions, with code you can run.

7 modules

30 runnable examples

7 exercises



YOUR HOST

Vishal Singh

Cloud Engineer

7+ years building cloud platforms on Azure and Kubernetes, and bringing agentic AI into real products.

CERTIFICATIONS

- Azure Solutions Architect Expert
- Agentic AI Business Solutions Architect
- Azure AI Apps and Agents Developer Associate
- Azure AI Engineer Associate
- Azure Developer Associate
- AI Transformation Leader
- Certified ScrumMaster (Scrum Alliance)

github.com/CodeMitVishal/python-foundations

SESSION MAP

Seven modules, each with live demos and an exercise

#	Module	Demos you build	Code folder
1	Data Types and Input and Output	Tax calculator, age calculator	01_data_types_io
2	Conditionals and Imports	Rock, paper, scissors game	02_conditionals_imports
3	Lists and Loops	Expense totals, loan calculator	03_lists_loops
4	Dictionaries, JSON, and Pip	Movie schedule, contacts, weather API	04_dicts_json_pip
5	Functions	Dice rolling game	05_functions
6	Classes and Objects	Company payroll	06_classes
7	Working with Files and Exceptions	File organizer	07_files_exceptions

github.com/CodeMitVishal/python-foundations

MODULE

00

Setup: terminal, VS Code, Git

The tools every developer uses, before the first line of Python.

IN THIS MODULE

The terminal and VS Code	`01_terminal_and_vscode.md`
Git and GitHub	`02_git_and_github.md`
Check your setup	`check_setup.py`
Exercise: first Pull Request	`ex00_first_pull_request.md`

LESSON · THE TERMINAL

10 commands you'll use every day

COMMAND	WHAT IT DOES
`pwd`	where am I?
`ls`	what's in this folder?
`cd projects` / `cd ..`	go into a folder / go up
`mkdir course`	make a folder
`code .`	open this folder in VS Code
`python hello.py`	run a Python file
Tab · **↑** · **Ctrl + C**	auto-complete · repeat · stop a program

```
$ python 00_setup_git/check_setup.py
  [OK      ] Python 3.10 or newer
  [OK      ] Git installed
  [OK      ] Git user.name set
You're ready to start Module 1!
```

> VS Code: install the **Python** extension, then **Python: Select Interpreter**.

LESSON · GIT AND GITHUB

Save snapshots, share your work

```
git config --global user.name "Your Name"     # once

git status                          # what changed?
git add hello.py                    # stage it
git commit -m "Add greeting"        # save a snapshot
git push                            # upload to GitHub
git pull                            # download others' work

git switch -c my-feature            # work on a branch
git push -u origin my-feature       # then open a Pull Request
```

> **add → commit → push**: the everyday loop.

> **Branches** keep `main` safe; a **Pull Request** asks for review and merge.

> Never commit passwords, `.env` or `.venv/`: use **.gitignore**.

EXERCISE 0 · YOUR TURN

Your first Pull Request

The task

1	Fork CodeMitVishal/python-foundations
2	git switch -c hello-<your-name>
3	Add submissions/<your-name>/hello.py
4	Commit, push, and open a Pull Request

```
print("Hi, I'm Vishal and I'm learning
      Python to build AI apps!")
```

Starter:
exercises/ex00_first_pull_request.md

> Done? Open a **Pull Request** and I'll review your code.

MODULE

01

Data Types and Input and Output

5 lessons + 1 exercise

`01_data_types_io/`

IN THIS MODULE	CODE
Introducing Data Types	`01_data_types.py`
Demo: Install Python	`hello.py`
Demo: Create a Tax Calculator	`02_tax_calculator.py`
Strings and Input and Output	`03_strings_input_output.py`
Demo: Age Calculator	`04_age_calculator.py`
Exercise: Tip calculator	`ex01_tip_calculator.py`

github.com/CodeMitVishal/python-foundations

LESSON · INTRODUCING DATA TYPES

Every value has a type

42
int · whole numbers

4.99
float · decimals

"hi"
str · text in quotes

True
bool · True or False

```
>>> length = 10
>>> type(length)
<class 'int'>
>>> type("10")
<class 'str'>
>>> 4.99 * 10
49.900000000000006
```

Variables store a value under a name. Read = as “gets”, not “equals”.

Names use lowercase_with_underscores and can’t start with a digit.

Floats are approximate, hence the trailing …006.

github.com/CodeMitVishal/python-foundations

DEMO · INSTALL PYTHON

Install Python, then say hello

1 · Download

Get Python 3 from python.org. On Windows, tick “Add python.exe to PATH”.

2 · Check

In a terminal, run python --version (or python3 on macOS).

3 · Run

Try lines at the >>> prompt, or save a .py file and run it.

```
$ python --version
Python 3.13.1
$ python
>>> print("Hello, world!")
Hello, world!
>>> exit()
$ python hello.py
```

The >>> prompt (REPL) runs one line at a time and shows the result. Perfect for experiments.

A .py file holds a program you keep, edit and run again.

DEMO · CREATE A TAX CALCULATOR

Arithmetic in a real program

```
# 02_tax_calculator.py
price = 59.99
tax_rate = 0.08  # 8%

tax = price * tax_rate
total = price + tax

print("Tax:", round(tax, 2))
print("Total:", round(total, 2))
```

```
Tax: 4.8
Total: 64.79
```

Arithmetic operators

Op	Means	Example
+	add	7 + 2 → 9
-	subtract	7 - 2 → 5
*	multiply	7 * 2 → 14
/	divide	7 / 2 → 3.5
//	floor divide	7 // 2 → 3
%	remainder	7 % 2 → 1
**	power	7 ** 2 → 49

github.com/CodeMitVishal/python-foundations

LESSON · STRINGS AND INPUT AND OUTPUT

Talk to the user with input() and print()

```
name = input("What's your name? ")
age = input("How old are you? ")

# input() always gives back a str
age = int(age)

print("Hi " + name + "!")
print(f"Next year you'll be {age + 1}.")
print(name.upper(), len(name))
```

SAMPLE RUN

```
What's your name? Vishal
How old are you? 30
Hi Vishal!
Next year you'll be 31.
VISHAL 6
```

input() always returns a str. Convert with int() or float() before doing maths.

Build text with + or with an f-string, which drops values right in.

DEMO · AGE CALCULATOR

Turn a birth year into an age

```
current_year = int(input("Current year: "))
birth_year = int(input("Birth year: "))

age = current_year - birth_year
months = age * 12
days = age * 365

print(f"You are about {age} years old.")
print(f"That's {months} months, or ~{days} days.")
```

SAMPLE RUN

```
Current year: 2026
Birth year: 1998
You are about 28 years old.
That's 336 months, or ~10220 days.
```

> **Wrap input() in int().** Without it you'd subtract one str from another, and Python raises a TypeError.

github.com/CodeMitVishal/python-foundations

EXERCISE 1 · YOUR TURN

Build a tip calculator

The task

1. Ask for the bill total, the tip % and how many people are paying.
2. Work out the tip, the total and each person's share.
3. Print all three with 2 decimals.

Hints

- input() gives you a str: convert it with float() or int().
- f"{tip:.2f}" shows exactly 2 decimals.

SAMPLE RUN

```
Bill total: 120
Tip %: 15
Split between how many people? 4
Tip: 18.00
Total: 138.00
Each person pays: 34.50
```

Starter · exercises/
`ex01_tip_calculator.py`

Solution · solutions/
`ex01_tip_calculator.py`

github.com/CodeMitVishal/python-foundations

MODULE

02

Conditionals and Imports

5 lessons + 1 exercise

`02_conditionals_imports/`

IN THIS MODULE	CODE
Conditionals	`01_conditionals.py`
Conditional Operators and Booleans	`02_operators_booleans.py`
Demo: Rock, Paper, Scissors Game	`03_rock_paper_scissors.py`
Import Python Modules	`04_imports_demo.py`
Demo: Randomize the Game	`05_rock_paper_scissors_random.py`
Exercise: Guess the number	`ex02_guess_the_number.py`

github.com/CodeMitVishal/python-foundations

LESSON · CONDITIONALS

Make decisions with if, elif and else

```
temp = int(input("Temperature in °C: "))

if temp >= 30:
    print("Hot. Drink water.")
elif temp >= 15:
    print("Nice weather.")
else:
    print("Bring a jacket.")

print("Done.")  # runs every time
```

Top to bottom. Python runs only the first block whose condition is True, then skips the rest.

Colon and indent. The : opens a block; the lines indented 4 spaces belong to it.

Optional parts. elif and else can be left out, and you can chain as many elif branches as you need.

Every comparison answers True or False

Comparison operators

Op	Means	Example
==	equal to	5 == 5 → True
!=	not equal to	5 != 3 → True
<	less than	2 < 1 → False
>	greater than	9 > 4 → True
<=	at most	3 <= 3 → True
>=	at least	2 >= 4 → False

Logical operators

Op	True when	Example
and	both sides are	age >= 18 and has_id
or	either side is	day == "Sat" or day == "Sun"
not	it flips the value	not is_raining

= assigns, == compares. Mixing them up is the classic first bug. True and False are capitalised.

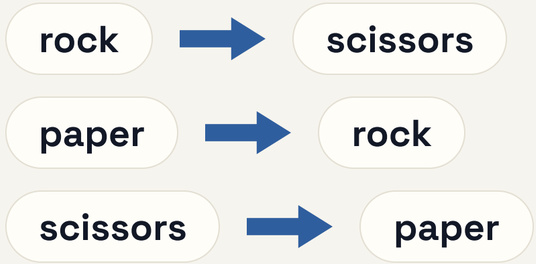
DEMO · ROCK, PAPER, SCISSORS GAME

Rock, paper, scissors with if and elif

```
you = input("rock, paper or scissors? ").lower()
cpu = "paper"  # hard-coded for now

if you == cpu:
    print("Tie!")
elif (you == "rock" and cpu == "scissors"
      or you == "paper" and cpu == "rock"
      or you == "scissors" and cpu == "paper"):
    print("You win!")
else:
    print("Computer wins!")
```

WHO BEATS WHOM



Arrow means “beats”.

> **and runs before or**, so each line is one winning pair. The ( ) let the test span lines.

github.com/CodeMitVishal/python-foundations

LESSON · IMPORT PYTHON MODULES

Borrow ready-made code with import

```
import random
random.randint(1, 6)        # 1 to 6, inclusive

import math
math.sqrt(81)               # 9.0

from datetime import date
date.today().year           # e.g. 2026

import statistics as stats
stats.mean([3, 5, 10])      # 6
```

A module is a file of ready-made code. Python ships with hundreds: the standard library.

import math → call math.sqrt(). **from math import sqrt** → call sqrt() directly.

as gives a module a short nickname. Keep imports at the top of the file.

Third-party packages come later, with pip, in Module 4.

DEMO · RANDOMIZE ROCK, PAPER, SCISSORS GAME

Let the computer choose with random

```python
import random

options = ["rock", "paper", "scissors"]
cpu = random.choice(options)
print("Computer chose", cpu)

you = input("rock, paper or scissors? ").lower()

# ...then the same if / elif / else as before
```

> **Only two lines changed.** The game logic stays the same; cpu just gets a new value on every run.

Handy random tools

Call	Gives you
choice(seq)	one random item
randint(a, b)	a whole number, a to b
random()	a float, 0.0 to 1.0
shuffle(lst)	the list, mixed in place

Each is called as random.name(...)

EXERCISE 2 · YOUR TURN

Build a guess-the-number game

The task

1. Pick a secret whole number from 1 to 10 with the random module.
2. Ask the player for one guess.
3. Print Correct!, Too low or Too high, then reveal the number.
4. Bonus: print “Out of range” for guesses outside 1 to 10.

Hints

- random.randint(1, 10) includes both 1 and 10.
- 1 <= guess <= 10 checks a range in one go.

TWO SAMPLE RUNS

```
Guess a number from 1 to 10: 4
Too low. It was 7.

Guess a number from 1 to 10: 7
Correct!
```

Starter · exercises/
`ex02_guess_the_number.py`

Solution · solutions/
`ex02_guess_the_number.py`

github.com/CodeMitVishal/python-foundations

MODULE

03

Lists and Loops

4 lessons + 1 exercise

`03_lists_loops/`

IN THIS MODULE	CODE
Lists and Loops	`01_lists_loops.py`
Demo: Sum Expenses	`02_sum_expenses.py`
Loops with range()	`03_range_loops.py`
Demo: Loan Payment Calculator	`04_loan_calculator.py`
Exercise: Grade book	`ex03_grade_book.py`

github.com/CodeMitVishal/python-foundations

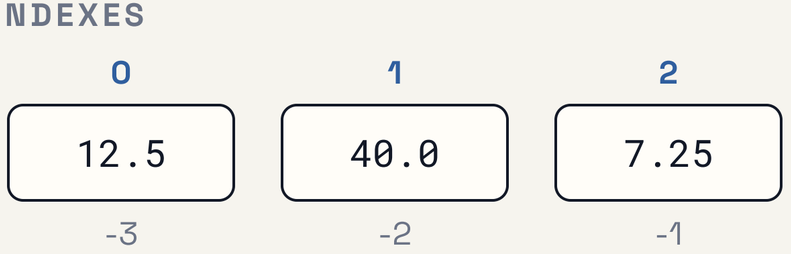
A list holds many values, in order

```
expenses = [12.50, 40.00, 7.25]

expenses[0]              # 12.5, the first item
expenses[-1]             # 7.25, the last item
len(expenses)            # 3

expenses.append(20)      # add to the end
expenses.remove(40.00)   # remove by value
print(expenses)          # [12.5, 7.25, 20]

for amount in expenses:
    print(amount)
```



> **Counting starts at 0.** Negative indexes count back from the end.

> **Lists can change:** append, insert, remove, pop and sort edit them in place.

> **for item in list** visits each item once, in order.

DEMO · SUM EXPENSES

Add up a week of spending with a loop

```
expenses = [10.50, 8.00, 5.00, 15.00,
            20.00, 20.00, 9.00]

total = 0
for expense in expenses:
    total += expense  # total = total + expense

print("You spent", total, "this week")
print("Average per day:", total / len(expenses))
print("Built-in sum():", sum(expenses))
```

OUTPUT

```
You spent 87.5 this week
Average per day: 12.5
Built-in sum(): 87.5
```

Accumulator pattern: start at 0, add inside the loop, use the result after it.

Built-ins sum(), min(), max() and len() do common list jobs in one call.

github.com/CodeMitVishal/python-foundations

LESSON · LOOPS WITH RANGE()

range() counts for you

Call	Numbers produced
range(5)	0 1 2 3 4
range(1, 6)	1 2 3 4 5
range(0, 10, 2)	0 2 4 6 8
range(5, 0, -1)	5 4 3 2 1

The stop value is never included. range(start, stop, step): start defaults to 0, step to 1.

```
for month in range(1, 13):
    print("Month", month)

names = ["Vishal", "Raj", "Li"]
for i in range(len(names)):
    print(i, names[i])
```

> **Use range()** when you need a fixed number of repeats, or the position of each item.

DEMO · LOAN PAYMENT CALCULATOR

Pay off a loan, one month at a time

```
balance = 1000.00
rate = 0.02         # 2% interest per month
payment = 100.00

for month in range(1, 25):
    balance = balance * (1 + rate) - payment
    if balance <= 0:
        print(f"Paid off in month {month}!")
        break
    print(f"Month {month}: ${balance:.2f} left")
```

OUTPUT

```
Month 1: $920.00 left
Month 2: $838.40 left
…
Month 11: $26.50 left
Paid off in month 12!
```

> **Each month:** add interest, then subtract the payment.

> **break** ends the loop early, as soon as the balance reaches zero.

github.com/CodeMitVishal/python-foundations

BONUS · WHILE, BREAK, CONTINUE

Loop until something changes

```
savings, months = 0, 0
while savings < 1000:           # checked before every round
    savings += 150
    months += 1                 # 7 months

while True:
    answer = input("Number (q to quit): ")
    if answer == "q":
        break                   # leave the loop
    if not answer.isdigit():
        continue                # skip to the next round
    print(int(answer) * 2)
```

for: once per item. **while**: as long as a condition is True.

break leaves the loop · **continue** skips the rest of this round.

Endless loop? **Ctrl + C** stops it.

BONUS · SLICING, TUPLES, SETS

Take parts, fix values, remove duplicates

CODE	RESULT
`days[0:3]`	first 3 (stop is not included)
`days[-2:]`	the last 2
`days[::-1]`	reversed copy
`lat, lon = (48.37, 10.89)`	a tuple, unpacked
`set(visitors)`	unique items only
`a & b` · `a | b`	in both · in either

list = ordered + changeable · **tuple** = fixed · **set** = unique + fast lookups

EXERCISE 3 · YOUR TURN

Build a grade book

The task

1. Use range() to ask for 5 test scores and store them in a list.
2. Print the average (1 decimal), the highest and the lowest score.
3. Count how many scores are 50 or more: those passed.

Hints

- sum(), max(), min() and len() all work on lists.
- Start a counter at 0 and add 1 inside the loop.

SAMPLE RUN

```
Score 1: 72
…
Score 5: 63
Average: 63.6
Highest: 88.0
Lowest: 45.0
Passed: 4 of 5
```

Starter · exercises/
ex03_grade_book.py

Solution · solutions/
ex03_grade_book.py

github.com/CodeMitVishal/python-foundations

MODULE

04

Dictionaries, JSON, and Pip

7 lessons + 1 exercise

`04_dicts_json_pip/`

IN THIS MODULE	CODE
Dictionaries	`01_dictionaries.py`
Demo: Movie Schedule Dictionary	`02_movie_schedule.py`
Combining Lists and Dictionaries	`03_lists_and_dicts.py`
Demo: Nested Contacts Dictionary	`04_nested_contacts.py`
Reading JSON and Installing with Pip	`05_json_basics.py`
Demo: Python Virtual Environment	`README.md`
Demo: Open Weather Map API	`06_weather_api.py`
Exercise: Word counter	`ex04_word_counter.py`

github.com/CodeMitVishal/python-foundations

LESSON · DICTIONARIES

A dictionary maps keys to values

```
movie = {
    "title": "Inception",
    "year": 2010,
    "rating": 8.8,
}

movie["title"]              # 'Inception'
movie["rating"] = 9.0       # change a value
movie["genre"] = "sci-fi"   # add a new key
movie.get("director", "?")  # '?' when missing

for key, value in movie.items():
    print(key, "->", value)
```

	List	Dict
Syntax	[ ]	{ }
Find by	position	key
Example	items[0]	movie["title"]

> **Keys are unique.** Assigning to an existing key replaces its value.

> **in checks keys:** "year" in movie → True. A missing key with [ ] raises KeyError; .get() doesn't.

DEMO · CREATE A MOVIE SCHEDULE DICTIONARY

A schedule: showtimes by movie

```
schedule = {
    "Dune": ["13:00", "16:30", "20:00"],
    "Wicked": ["12:15", "18:45"],
    "Up": ["10:00"],
}

movie = input("Which movie? ")
if movie in schedule:
    times = schedule[movie]
    print(f"{movie} plays at {', '.join(times)}")
else:
    print("Not showing today.")
```

SAMPLE RUN

```
Which movie? Dune
Dune plays at 13:00, 16:30, 20:00
```

> **Values can be anything.** Here each movie maps to a list of showtimes.

> **in** checks the key before you look it up, so a typo can't crash the program.

> **", ".join(list)** glues a list of strings into one.

LESSON · COMBINING LISTS AND DICTIONARIES

Nest lists and dicts to model real data

List of dicts: many records

```
people = [
    {"name": "Vishal", "age": 31},
    {"name": "Raj", "age": 27},
]
people[1]["name"]  # 'Raj'

for person in people:
    print(person["name"])
```

Dict of lists: grouped values

```
grades = {
    "math": [90, 85],
    "art": [70, 95, 88],
}
grades["art"][0]  # 70

grades["math"].append(77)
```

> **Read nested access left to right:** people[1] gives a dict, then ["name"] gives that dict's value.

DEMO · PARSE A NESTED CONTACTS DICTIONARY

Dig values out of nested contacts

```
contacts = {
    "Vishal": {"phone": "555-0101",
               "tags": ["work", "gym"]},
    "Raj": {"phone": "555-0199",
            "tags": ["family"]},
}

for name, info in contacts.items():
    tags = ", ".join(info["tags"])
    print(f"{name}: {info['phone']} ({tags})")

contacts["Vishal"]["tags"][1] # 'gym'
```

OUTPUT

```
Vishal: 555-0101 (work, gym)
Raj: 555-0199 (family)
```

ONE STEP AT A TIME

`contacts["Vishal"]` → Vishal's dict

`["tags"]` → the list ["work", "gym"]

`[1]` → "gym"

JSON: data as text that maps onto dicts

```
import json

text = '{"name": "Vishal", "langs": ["Python", "SQL"]}'
data = json.loads(text)       # str -> dict
data["langs"][0]              # 'Python'
back = json.dumps(data)       # dict -> str

with open("contacts.json") as f:
    contacts = json.load(f)  # file -> dict
```

> **The s means string:** loads and dumps work on str; load and dump work on open files.

JSON → Python

JSON	Python
object { }	dict
array [ ]	list
string	str
number	int or float
true / false	True / False
null	None

DEMO · CREATE A PYTHON VIRTUAL ENVIRONMENT

One project, one virtual environment

```
# 1. create and activate a venv
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux

# 2. install packages into it
pip install requests
pip list
pip freeze > requirements.txt

# 3. leave it when you're done
deactivate
```

A venv is a private folder of packages for one project, so versions never clash.

pip downloads packages from PyPI, the Python Package Index.

requirements.txt lists them; pip install -r requirements.txt rebuilds the setup anywhere.

(.venv) appears in your prompt while it is active.

DEMO · USE THE OPEN WEATHER MAP API

Ask a web API for today's weather

```
import requests

API_KEY = "your-api-key"
BASE = "https://api.openweathermap.org"
url = BASE + "/data/2.5/weather"
params = {"q": "Berlin", "appid": API_KEY,
          "units": "metric"}

response = requests.get(url, params=params)
data = response.json()  # JSON -> dict
print(data["name"], data["main"]["temp"], "°C")
print(data["weather"][0]["description"])
```

RESPONSE, TRIMMED

```
{"name": "Berlin",
 "main": {"temp": 12.4},
 "weather": [
   {"description": "light rain"}]}
```

Example values

> **requests** isn't built in: pip install it inside your venv first.

> **Protect your key.** Don't share code with a real key in it; load it from an environment variable.

EXERCISE 4 · YOUR TURN

Count the words in a sentence

The task

1. Ask for a sentence.
2. Count each word in a dictionary, ignoring case and . , ! ?
3. Print the dictionary as pretty JSON.
4. Print the most common word.

Hints

- sentence.lower().split() gives you a list of words.
- word.strip(".,!?") trims punctuation from both ends.
- json.dumps(counts, indent=2) pretty-prints a dictionary.

SAMPLE RUN

```
Type a sentence: Go team, go!
{
  "go": 2,
  "team": 1
}
Most common: go (2x)
```

Starter · exercises/
`ex04_word_counter.py`

Solution · solutions/
`ex04_word_counter.py`

github.com/CodeMitVishal/python-foundations

MODULE

05

Functions

2 lessons + 1 exercise

`05_functions/`

IN THIS MODULE	CODE
Functions	`01_functions.py`
Demo: Dice Rolling Game with Functions	`02_dice_game.py`
Exercise: Temperature converter	`ex05_temperature_converter.py`

github.com/CodeMitVishal/python-foundations

LESSON · FUNCTIONS

Name a job once, run it anywhere

```
def greet(name, greeting="Hello"):
    """Return a friendly greeting."""
    message = f"{greeting}, {name}!"
    return message

greet("Vishal")                    # 'Hello, Vishal!'
greet("Raj", "Hi")                 # 'Hi, Raj!'
greet(greeting="Hey", name="Li")   # 'Hey, Li!'

# message only exists inside greet()
print(message)  # NameError
```

def name(parameters): defines it. The indented body runs only when you call it.

return hands a value back. No return means the call gives None.

Defaults make a parameter optional; keyword arguments can come in any order.

Local scope: variables made inside a function stay inside it.

DEMO · DICE ROLLING GAME WITH FUNCTIONS

A dice game built from small functions

The functions

```
import random

def roll_die(sides=6):
    return random.randint(1, sides)

def play_round(player):
    total = roll_die() + roll_die()
    print(f"{player} rolled {total}")
    return total
```

The game

```
you = play_round("You")
cpu = play_round("CPU")

if you > cpu:
    print("You win!")
elif you < cpu:
    print("CPU wins!")
else:
    print("Draw.")
```

> **One job per function:** change sides=6 once and every roll in the game uses the new die.

github.com/CodeMitVishal/python-foundations

EXERCISE 5 · YOUR TURN

Write a temperature converter

The task

1. Write c_to_f(celsius) and f_to_c(fahrenheit), each returning the result.
2. Write convert(value, unit="C") that returns a sentence using them.
3. Ask for a temperature and a unit, then print convert(value, unit).

Hints

- F = C × 9 / 5 + 32, and C = (F − 32) × 5 / 9
- The default value makes unit optional, so convert(100) works too.

SAMPLE RUN

```
Temperature: 100
Unit (C or F): c
100.0°C = 212.0°F
```

Starter · exercises/
`ex05_temperature_converter.py`

Solution · solutions/
`ex05_temperature_converter.py`

github.com/CodeMitVishal/python-foundations

MODULE

06

Classes and Objects

4 lessons + 1 exercise

`06_classes/`

IN THIS MODULE	CODE
Classes	`01_employee.py`
Demo: Company Payroll with Classes	`02_payroll.py`
Class Inheritance	`03_inheritance.py`
Demo: Payroll with Inheritance	`04_payroll_inheritance.py`
Exercise: Bank accounts	`ex06_bank_account.py`

github.com/CodeMitVishal/python-foundations

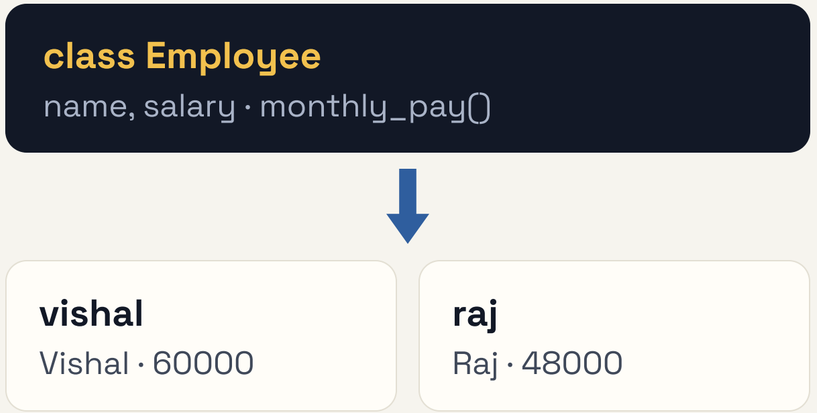
LESSON · CLASSES

A class is a blueprint for objects

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def monthly_pay(self):
        return self.salary / 12

vishal = Employee("Vishal", 60000)
raj = Employee("Raj", 48000)
vishal.name           # 'Vishal'
vishal.monthly_pay()  # 5000.0
raj.monthly_pay()     # 4000.0
```



__init__ runs when you create an object and sets its attributes.

self is the object itself; every method gets it first.

github.com/CodeMitVishal/python-foundations

LESSON · CLASSES AND OBJECTS

Class vs object: the standard definitions

CLASS

A **class** is a **user-defined data type**: a blueprint that defines the **attributes** (data) and **methods** (behaviour) that its objects will have.

You write it once with the `class` keyword. On its own it describes things; it isn't one of them.

📐 Like the blueprint of a house

OBJECT

An **object** is an **instance of a class**: a real thing built from the blueprint and stored in memory with its own values.

Every object has an **identity** (`id()`), a **type** (its class) and a **state** (its attribute values).

🏠 Like a house built from that blueprint

```
class Employee:                  # the blueprint
    ...
vishal = Employee("Vishal", 60000)  # an object
type(vishal)                     # <class 'Employee'>
isinstance(vishal, Employee) # True
```

> **Instantiation** = creating an object from a class: `Employee(...)`

> One class, **many objects**: each object keeps its own data but shares the class's methods.

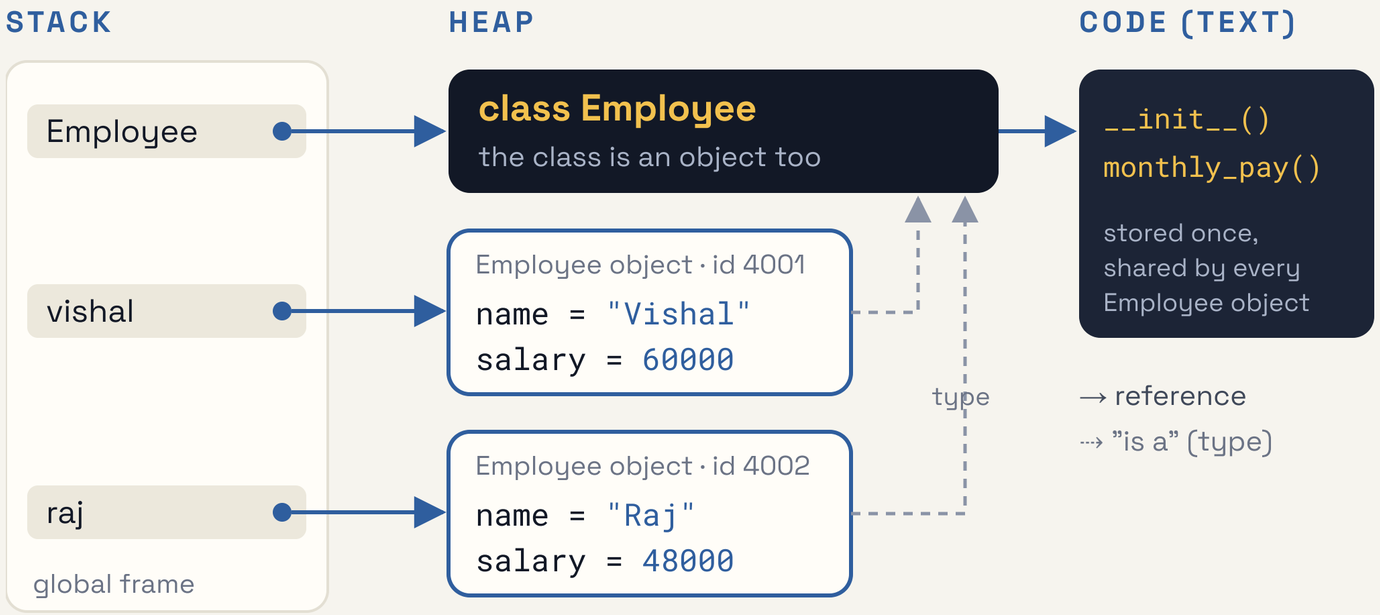
LESSON · OBJECTS IN MEMORY

Where Python keeps your objects: stack, heap and code

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def monthly_pay(self):
        return self.salary / 12

vishal = Employee("Vishal", 60000)
raj = Employee("Raj", 48000)
```



Stack: names

Each running function gets a frame of names that **refer** to objects. The frame is removed when the function returns.

Heap: objects

Every object (ints, strings, lists, your Employees) lives here as long as something refers to it.

Code (text): instructions

The compiled code (bytecode) of functions and methods: one copy, shared by all objects.

LESSON · CREATING AN OBJECT

What happens when you run `vishal = Employee("Vishal", 60000)`

1 · HEAP

A new, empty Employee object is created on the heap.

Python's `__new__` does this for you.

2 · STACK

__init__ is called. A new frame goes on the stack: `self` → the new object, `name` → "Vishal", `salary` → 60000.

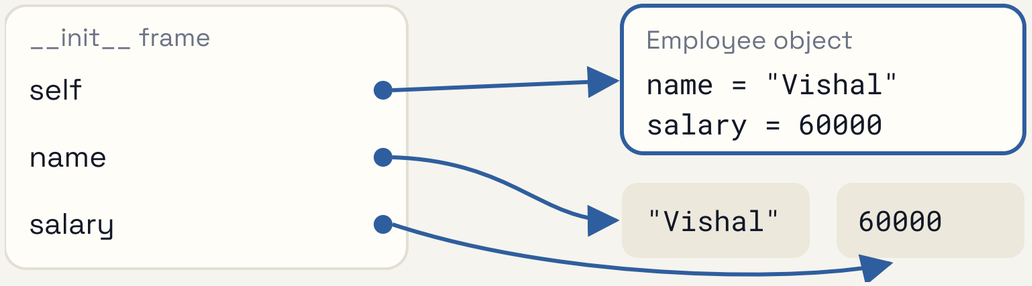
3 · HEAP

self.name = name and **self.salary = salary** store the values **inside the object**.

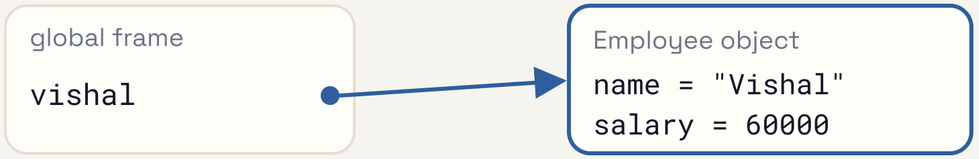
4 · STACK

__init__ returns and its frame is removed. The name `vishal` now refers to the object.

DURING __init__ (STEP 2-3)



AFTER (STEP 4)



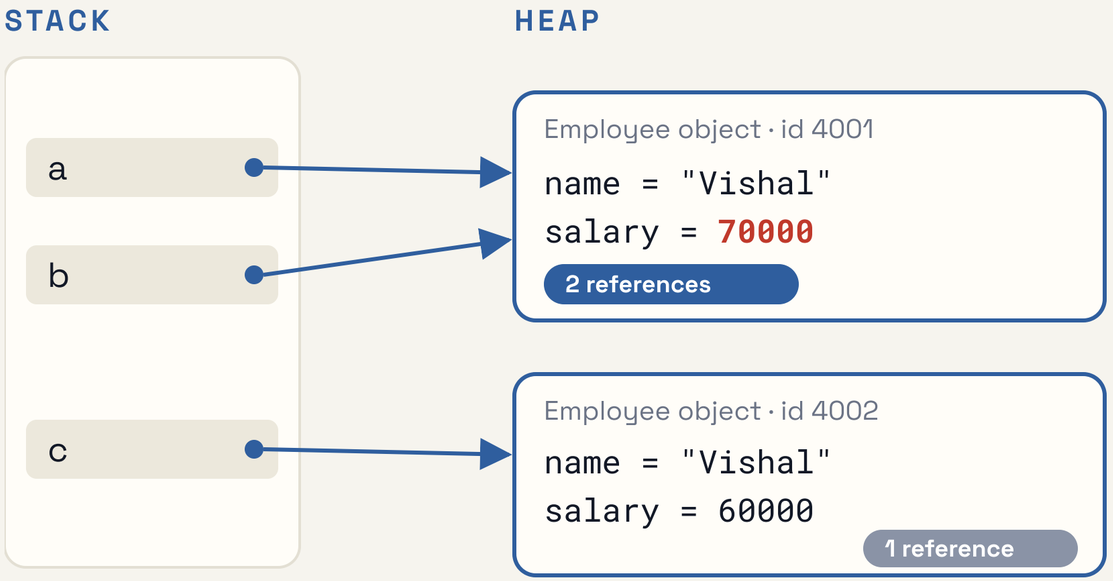
The __init__ frame is gone; the object stays on the heap because `vishal` still refers to it.

LESSON · REFERENCES

Variables hold references, not the object itself

```
a = Employee("Vishal", 60000)
b = a                          # same object, 2 names
c = Employee("Vishal", 60000)  # new object

b.salary = 70000
print(a.salary)  # 70000 (changed via b!)
print(a is b)      # True  same object
print(a is c)      # False different objects
print(id(a) == id(b))  # True
```



= copies the reference

`b = a` doesn't copy the object. Both names point at the same one, so a change through `b` shows in `a`.

is vs ==

`is` asks "same object?" (same `id`). `==` asks "equal values?"

Garbage collection

When no name refers to an object any more, Python frees its heap memory automatically. `del a` removes the name, not the object.

DEMO · COMPANY PAYROLL WITH CLASSES

A Company object that holds Employees

```
class Company:
    def __init__(self, name):
        self.name = name
        self.employees = []

    def hire(self, employee):
        self.employees.append(employee)

    def payroll(self):
        total = 0
        for e in self.employees:
            total += e.monthly_pay()
        return total
```

```
acme = Company("Acme")
acme.hire(Employee("Vishal", 60000))
acme.hire(Employee("Raj", 48000))
print(acme.payroll())  # 9000.0
```

Objects hold objects. A Company keeps a list of Employee objects.

Ask, don't calculate. payroll() asks each employee for monthly_pay() and adds the answers.

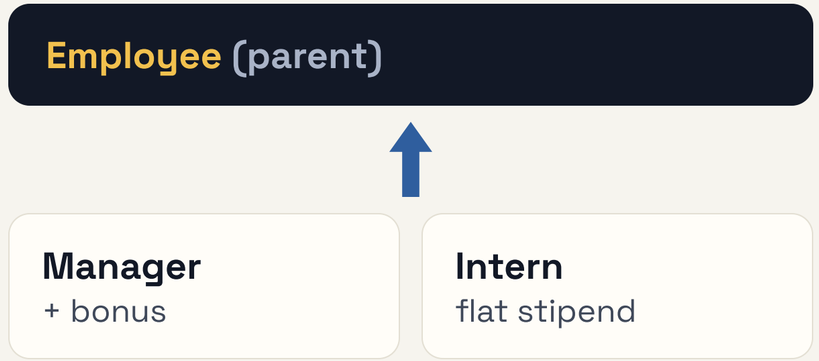
LESSON · CLASS INHERITANCE

Inherit a class, then specialise it

```
class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus

    def monthly_pay(self):  # override
        base = super().monthly_pay()
        return base + self.bonus / 12

boss = Manager("Li", 90000, 12000)
boss.name                       # 'Li', inherited
boss.monthly_pay()              # 8500.0
isinstance(boss, Employee)      # True
```



A child class gets every attribute and method of its parent.

Override by redefining a method; super() calls the parent's version.

github.com/CodeMitVishal/python-foundations

DEMO · COMPANY PAYROLL WITH INHERITANCE

Mixed staff, one payroll loop

```
class Intern(Employee):
    def monthly_pay(self):
        return 1200  # flat stipend

acme = Company("Acme")
acme.hire(Employee("Vishal", 60000))
acme.hire(Manager("Li", 90000, 12000))
acme.hire(Intern("Sam", 0))

for e in acme.employees:
    print(e.name, e.monthly_pay())
print("Total:", acme.payroll())
```

OUTPUT

```
Vishal 5000.0
Li 8500.0
Sam 1200
Total: 14700.0
```

> **Polymorphism:** one call, monthly_pay(), and each class gives its own answer.

> **Company never changed.** New kinds of staff plug straight in.

github.com/CodeMitVishal/python-foundations

EXERCISE 6 · YOUR TURN

Model bank accounts with classes

The task

1. BankAccount(owner, balance=0) with deposit() and withdraw().
2. withdraw() refuses when the balance is too low.
3. describe() returns a line such as “Vishal: 150.00”.
4. SavingsAccount(BankAccount) adds a rate and add_interest().

Hints

- super().__init__(owner, balance) reuses the parent’s setup.
- The starter file includes test code to paste in.

EXPECTED OUTPUT

```
Declined: Vishal has only 150.00
Vishal: 150.00
Raj: 1050.00
```

Starter · exercises/
`ex06_bank_account.py`

Solution · solutions/
`ex06_bank_account.py`

MODULE

07

Working with Files and Exceptions

5 lessons + 1 exercise

`07_files_exceptions/`

IN THIS MODULE	CODE
Exceptions	`01_exceptions.py`
Reading Files	`02_reading_files.py`
Demo: Writing Files	`03_writing_files.py`
File Manipulation	`04_file_manipulation.py`
Demo: File Organization	`05_file_organizer.py`
Exercise: Notes app	`ex07_notes_app.py`

github.com/CodeMitVishal/python-foundations

LESSON · EXCEPTIONS

Catch errors with try and except

```
try:
    age = int(input("Age: "))
    per_year = 1000 / age
except ValueError:
    print("Please type a whole number.")
except ZeroDivisionError:
    print("Age can't be zero.")
else:
    print(f"{per_year:.1f} per year")
finally:
    print("Thanks!")
```

> **else** runs only when nothing failed; **finally** runs every time.

Errors you'll meet first

Exception	Caused by
ValueError	int("abc")
TypeError	"2" + 2
KeyError	d["missing"]
IndexError	[1, 2][5]
FileNotFoundError	open("nope.txt")
ZeroDivisionError	1 / 0

LESSON · READING FILES

Read a file three ways with open()

```
with open("notes.txt") as f:
    text = f.read()          # whole file, one str

with open("notes.txt") as f:
    for line in f:           # one line at a time
        print(line.strip())

with open("notes.txt") as f:
    lines = f.readlines()    # a list of lines
```

with closes the file for you, even if an error happens inside.

Paths are relative to the folder you run the script from.

encoding="utf-8" keeps accents and emoji intact.

No such file? You get FileNotFoundError. Catch it with try / except.

DEMO · WRITING FILES

Write and append with file modes

```
shopping = ["milk", "eggs", "bread"]

with open("list.txt", "w") as f:  # overwrite
    for item in shopping:
        f.write(item + "\n")

with open("list.txt", "a") as f:  # add to the end
    f.write("coffee\n")

with open("list.txt") as f:
    print(f.read())
```

> **write() adds no newline.** Put "\n" at the end of each line yourself.

The mode argument

Mode	What happens
"r"	read (the default); error if missing
"w"	write; creates the file or wipes it
"a"	append; creates it or adds to the end
"x"	create; error if it already exists

github.com/CodeMitVishal/python-foundations

LESSON · FILE MANIPULATION

List, copy, move and delete files in code

```
import os
import shutil

os.listdir(".")                # names in a folder
os.path.exists("data.csv")     # True or False
os.makedirs("backup", exist_ok=True)

shutil.copy("data.csv", "backup/")
os.rename("old.txt", "new.txt")
shutil.move("new.txt", "archive/")
os.remove("temp.txt")          # no undo!
```

Or use pathlib

```
from pathlib import Path

p = Path("report.pdf")
p.name   # 'report.pdf'
p.stem   # 'report'
p.suffix # '.pdf'
```

> **os.remove()** skips the recycle bin. Try scripts on copies first.

DEMO · FILE ORGANIZATION

Sort a messy folder by file type

```python
import os
import shutil

folder = "Downloads"
for name in os.listdir(folder):
    path = os.path.join(folder, name)
    if not os.path.isfile(path):
        continue  # skip folders
    ext = os.path.splitext(name)[1].lower() or ".none"
    target = os.path.join(folder, ext[1:] + "_files")
    os.makedirs(target, exist_ok=True)
    shutil.move(path, target)
```

AFTER RUNNING IT

```
Downloads/
├─ pdf_files/
│   cv.pdf  scan.pdf
├─ jpg_files/
│   photo.jpg
└─ txt_files/
    notes.txt
```

> **continue** jumps to the next item, so folders are left alone.

> **Test on a copy first.** The moves happen for real.

BONUS · DEBUGGING

Read the traceback from the bottom up

```
Traceback (most recent call last):
  File "06_debugging.py", line 25, in <module>
    average_price([])
  File "06_debugging.py", line 20, in average_price    <- WHERE
    return total / len(prices)                          <- the line
ZeroDivisionError: division by zero                     <- WHAT

print(f"DEBUG {subtotal=} {tax=}")    # quick: show names + values
breakpoint()                          # pause: p var, n, s, c, q
```

> The **last line** says what went wrong; the lines above say where.

> VS Code: click left of a line number (red dot), press **F5**, hover over variables.

> Explain your code out loud: **rubber duck debugging** works.

EXERCISE 7 · YOUR TURN

Build a notes app that saves to a file

The task

1. Show a menu, 1) Add 2) Show 3) Quit, until the user quits.
2. Add appends the note to my_notes.txt.
3. Show prints each note with a number, or “No notes yet.”
4. Anything else prints “Please type 1, 2 or 3.”

Hints

- Catch FileNotFoundError when the file doesn’t exist yet.
- int(”x”) raises ValueError, so wrap it in try / except.

SAMPLE RUN

```
1) Add  2) Show  3) Quit: 1
Note: Buy milk
Saved.
1) Add  2) Show  3) Quit: 2
1. Buy milk
1) Add  2) Show  3) Quit: 3
```

Starter · exercises/
`ex07_notes_app.py`

Solution · solutions/
`ex07_notes_app.py`

github.com/CodeMitVishal/python-foundations

CHEAT SHEET

The whole course on one slide

Concept	Syntax
Variable	x = 5
f-string	f”Hi {name}”
Decision	if x > 0: … elif … else: …
Loop	for i in range(3):
List	nums = [1, 2, 3]
Import	from random import choice

Concept	Syntax
Dictionary	d = {”key”: ”value”}
Function	def add(a, b): return a + b
Class	class Manager(Employee):
File	with open(”f.txt”) as f:
Errors	try: … except ValueError: …
Packages	pip install requests

github.com/CodeMitVishal/python-foundations

RECAP

What you can do after this course

01
Store values and talk to the user

02
Make decisions and import modules

03
Loop over lists and ranges

04
Model data with dicts and JSON, and call web APIs

05
Package logic into reusable functions

06
Design classes that share behaviour

07
Read, write and organise files safely

github.com/CodeMitVishal/python-foundations

KEEP PRACTISING

Three small projects that use it all

Expense tracker

Save expenses to a JSON file and print a monthly total and average.

Modules 3, 4, 7

Weather CLI

Ask for a city, call the API, and handle a bad city name with try / except.

Modules 4, 5, 7

Quiz game

A Question class, a list of questions, and high scores saved to a file.

Modules 5, 6, 7



Subscribe to CodeMitVishal

@CodeMitVishal · Code and exercises on GitHub

Want to support us? Buy me a coffee
buymeacoffee.com/codemitvishal

github.com/CodeMitVishal/python-foundations